

Dossier d'Analyse de Projet

Étude Comparative de Classificateurs pour la Base BDshape

Aidoudi Aaron

Année Universitaire 2025 - 2026

Table des matières

I.	Introduction	3
II.	Lecture des données	3
	A. Lecture	3
	B. Implémentation Python	4
III.	Classification par k-Nearest-Neighbors	5
	A. Méthodologie	5
	B. Implémentation	5
	C. Résultats	6
IV.	Classification par k-means	8
	A. Méthodologie	8
	B. Implémentation	8
	C. Résultats	9
V.	Classification par vote majoritaire	11
	A. Méthodologie	11
	B. Implémentation	11
	C. Résultats	12
VI.	Conclusions	13
VII.	Sources	14
	A. Références bibliographiques	14
	B. Ressources en ligne	14

I. Introduction

La **reconnaissance de formes** constitue un domaine fondamental de la vision par ordinateur, avec des applications variées allant de la numérisation de documents à l'identification biométrique. Ce projet s'inscrit dans cette thématique en proposant une étude comparative de différentes approches de classification appliquées au dataset **BDshape**, une base d'images binaires représentant neuf classes d'objets distincts.

Le dataset présente plusieurs défis caractéristiques des problèmes réels de reconnaissance de formes. Certaines classes contiennent des formes occultées, notamment les avions (*classe 4*) et les mains (*classe 5*), où des parties significatives de l'objet sont manquantes, tandis que d'autres présentent des représentations partielles, comme les lapins (*classe 2*) ou les silhouettes humaines (*classe 3*). Aussi, la classe des outils (*classe 6*) comporte des distorsions géométriques importantes, tandis que la classe des animaux (*classe 8*) regroupe des formes hétérogènes qui partagent une catégorie sémantique commune mais diffèrent fortement dans leur apparence visuelle. Ces particularités rendent la classification non triviale et permettent d'évaluer la robustesse des différentes méthodes dans des conditions variées.

Pour représenter ces formes, nous disposons de cinq **descripteurs** classiques issus de l'analyse d'images. Le descripteur **E34** capture l'ellipticité des formes à travers les moments d'ordre 2. **GFD** (*Generic Fourier Descriptor*) encode les contours dans le domaine fréquentiel en combinant des informations radiales et angulaires. **SA** (*Signature Angulaire*) mesure la distance entre le centroïde et le contour pour différentes orientations. Enfin, les descripteurs **F0** et **F2** évaluent respectivement la déformation sous une force constante et sous des forces directionnelles. Chacune de ces représentations offre une perspective différente sur la géométrie des formes, et leurs performances relatives constituent un axe d'analyse de ce projet.

L'objectif principal de ce travail est d'implémenter et comparer trois approches de **classification** : la méthode des k -plus-proches voisins **kNN**, un algorithme supervisé utilisé pour sa simplicité et son efficacité, l'algorithme des **k-means** (*nuées dynamiques*), une approche non supervisée permettant d'explorer la structure naturelle des données sans utiliser les étiquettes de classes, et une stratégie de **vote majoritaire** qui combine les prédictions issues des différents descripteurs pour tirer parti de leur complémentarité. Pour chaque approche, nous évaluerons les performances à l'aide de métriques telles que le taux de reconnaissance, la précision, le rappel et la F-mesure, tout en explorant l'impact des paramètres comme le nombre de voisins k ou la fonction de distance utilisée.

II. Lecture des données

A. Lecture

La première étape de ce projet a consisté à **charger** et **organiser** l'ensemble des données fournies pour le dataset **BDshape**. Chaque échantillon (*99 échantillons répartis équitablement en 9 classes d'objets*) est représenté par cinq méthodes de description différentes. Chaque fichier contient une valeur numérique par ligne, correspondant à une dimension du vecteur caractéristique. Cette structure a permis d'implémenter une fonction de lecture générique capable de traiter indifféremment tous les types de descripteurs.

Concernant les **dimensions des vecteurs caractéristiques**, nous avons observé que quatre méthodes sur cinq correspondent exactement aux spécifications du sujet. À propos du descripteur **GFD**, le sujet indique qu'il devrait s'agir d'un vecteur caractéristique de 36 valeurs. Cependant, nos fichiers contiennent systématiquement 100 valeurs pour cette méthode. Cette différence s'explique vraisemblablement par le fait que les fichiers fournis utilisent une configuration avec une résolution plus élevée que la description du sujet.

L'analyse statistique des données révèle des ordres de grandeur très différents entre les méthodes. Les descripteurs **E34**, **GFD** et **SA** contiennent des valeurs relativement petites (*moyennes arrondies respectives de 0.80, 0.05 et 0.01*), tandis que **F0** et **F2** présentent des valeurs nettement supérieures (*moyennes arrondies respectives de 95414 et 2248*). Ces disparités d'échelle suggèrent que la normalisation des caractéristiques sera très probablement nécessaire lors de l'implémentation des classificateurs.

La **distribution** des échantillons par classe est parfaitement **équilibrée** pour toutes les méthodes, ce qui facilitera l'évaluation des performances, en évitant les biais liés à des classes sur ou sous-représentées. L'ensemble des données

a été sauvegardé dans une structure **NumPy** nommée *donnees_chargees.npy* pour faciliter leur réutilisation lors des étapes suivantes du projet.

Le choix de stocker les données sous forme de tableaux NumPy plutôt que de listes Python standard se justifie par **l'efficacité** computationnelle, NumPy permettant d'effectuer des opérations mathématiques sur des tableaux entiers sans boucles explicites, ce qui est crucial pour optimiser les temps de calcul, notamment lors de l'évaluation des distances dans les algorithmes **k-NN** et **k-means**. Aussi, ces structures constituent un **standard** de l'écosystème scientifique Python.

B. Implémentation Python

Le programme Python développé pour cette étape repose sur deux modules complémentaires :

Le premier, nommé *charge_donnees.py*, encapsule toute la logique de chargement et d'organisation des données dans une classe dédiée appelée *ChargeDonnees*. Cette classe centralise l'ensemble des opérations nécessaires à la manipulation du dataset.

- Pour la lecture des vecteurs caractéristiques, la méthode *charger_feature_vector()* ouvre chaque fichier en mode lecture et extrait les valeurs numériques ligne par ligne. Elle convertit ensuite ces valeurs en un tableau NumPy.
- La méthode principale *charger_donnees()* orchestre le chargement complet du dataset : elle itère sur l'ensemble des combinaisons possibles de classes, d'échantillons et de méthodes pour construire un dictionnaire. Pour chaque méthode de description, ce dictionnaire associe trois éléments : une matrice NumPy contenant tous les vecteurs caractéristiques (de dimension $n \times d$ où n est le nombre d'échantillons et d le nombre de caractéristiques), un vecteur d'étiquettes de classes, et une liste de tuples identifiant chaque échantillon par sa classe et son numéro. Cette organisation facilite l'accès aux données lors de l'implémentation des classificateurs.
- Enfin, la méthode *calculer_stats()* calcule des statistiques descriptives globales pour chaque méthode. Elle produit un data frame rassemblant le nombre d'échantillons chargés, la dimensionnalité des vecteurs, ainsi que la moyenne, l'écart-type, le minimum et le maximum des valeurs rencontrées.

Le second module, *explore_donnees.py*, constitue le point d'entrée du processus de chargement. Il instancie un objet *ChargeDonnees* en spécifiant le chemin vers le répertoire des données, puis invoque successivement les différentes méthodes de la classe. L'exécution de ce script produit un rapport détaillé dont une capture est donnée ci-dessous.

```
Lecture des données :
```

```
1. Chargement des données :
E34: 99 échantillons chargés, dimension: (16,)
GFD: 99 échantillons chargés, dimension: (100,)
SA: 99 échantillons chargés, dimension: (90,)
F0: 99 échantillons chargés, dimension: (128,)
F2: 99 échantillons chargés, dimension: (128,)

2. Vérification de l'intégrité :
E34: 99/99 fichiers - OK
GFD: 99/99 fichiers - OK
SA: 99/99 fichiers - OK
F0: 99/99 fichiers - OK
F2: 99/99 fichiers - OK

3. Vérification des dimensions :
E34: dimension 16 (attendu: 16) - OK
GFD: dimension 100 (attendu: 36) - ?
SA: dimension 90 (attendu: 90) - OK
F0: dimension 128 (attendu: 128) - OK
F2: dimension 128 (attendu: 128) - OK

4. Statistiques descriptives :
```

Méthode	Nombre échantillons	Dimension	Moyenne	Écart-type	Min	Max
E34	99	16	0.801570	0.766071	0.166505	10.541180
GFD	99	100	0.046566	0.080600	0.000142	0.878278
SA	99	90	0.011111	0.005593	0.000000	0.147832
F0	99	128	95414.697595	60450.875547	2106.737527	367074.855047
F2	99	128	2248.876409	1052.169610	193.612867	6715.360890

```
5. Sauvegarde des données chargées :
Données sauvegardées dans 'donnees_chargees.npy'
```

Résultat de la lecture des données

III. Classification par k -Nearest-Neighbors

A. Méthodologie

La méthode des k -plus-proches voisins constitue notre première approche de **classification supervisée**. Le principe fondamental de celle-ci repose sur l'idée qu'un échantillon inconnu devrait appartenir à la même classe que ses voisins les plus proches dans l'espace des caractéristiques. L'algorithme ne réalise pas d'apprentissage proprement dit, la phase d'entraînement se limitant à mémoriser l'ensemble des échantillons d'apprentissage avec leurs étiquettes de classe.

Comme nous l'avons constaté dans la partie précédente, les cinq méthodes de description présentent des ordres de grandeur très différents. Nous avons donc opté pour l'approche de **normalisation z-score**, telle que pour chaque dimension, nous calculons la moyenne et l'écart-type sur l'ensemble d'entraînement, puis nous transformons les données en soustrayant la moyenne et en divisant par l'écart-type. Cette transformation assure que toutes les dimensions contribuent équitablement au calcul de distance, avec une moyenne de zéro et un écart-type de un.

Notons que ces paramètres de normalisation sont calculés uniquement sur l'ensemble d'entraînement, puis appliqués de manière identique aux données de test. Cette précaution évite toute fuite d'information du test vers l'entraînement, ce qui biaiserait les résultats vers le haut.

Le processus de **classification** d'un échantillon test se décompose en plusieurs étapes séquentielles. Après normalisation, nous calculons la **distance** entre cet échantillon et chacun des échantillons d'entraînement en utilisant la métrique de **Minkowski** choisie. Ces distances sont ensuite triées par ordre croissant, et nous sélectionnons les k échantillons présentant les distances les plus faibles. La décision finale s'effectue telle que nous comptons le nombre de voisins appartenant à chaque classe, et attribuons à l'échantillon test la classe la plus représentée parmi ses k voisins (*en cas d'égalité parfaite, la classe ayant le plus petit indice est sélectionnée*).

Le choix du paramètre k représente un compromis entre biais et variance. Une valeur égale à 1 produit un classificateur très sensible au bruit (*chaque échantillon de test est classé selon son unique plus proche voisin, ce qui peut conduire à des frontières de décision très irrégulières et à du sur-apprentissage*), là où à l'opposé, une valeur trop élevée lisse excessivement les frontières de décision (*et peut conduire à du sous-apprentissage, le modèle devenant trop simple pour capturer la vraie structure des données*). Dans notre cas, des valeurs comprises entre 3 et 10 semblent raisonnables. Nous avons choisi k égal à 5 comme configuration par défaut, valeur couramment utilisée dans la littérature et suffisamment grande pour atténuer l'effet du bruit tout en restant assez petite pour capturer les variations locales.

Concernant la **stratégie d'évaluation**, utiliser l'ensemble d'entraînement comme ensemble de test conduirait à une estimation trompeuse, le modèle reconnaîtrait simplement les exemples qu'il a déjà mémorisés.

Nous avons alors retenu la validation croisée **Leave-One-Out** comme protocole d'évaluation. Cette méthode, particulièrement adaptée aux petits ensembles de données, fonctionne selon le principe suivant : pour chaque échantillon du dataset, nous construisons un ensemble d'entraînement contenant tous les autres échantillons, et nous évaluons la prédiction du modèle sur l'unique échantillon exclu. Ce processus est ainsi répété 99 fois dans notre cas, de sorte que chaque échantillon serve exactement une fois comme donnée de test. L'estimation finale de la performance est obtenue en moyennant les résultats sur l'ensemble des itérations.

Cette approche présente plusieurs **avantages** décisifs pour notre contexte, tels que la **maximisation** de l'utilisation des données d'entraînement (*à chaque itération, le modèle s'entraîne sur 98 échantillons, crucial quand le dataset total ne contient que 99 éléments*), une estimation presque **non biaisée** (*chaque échantillon étant évalué tel que nouveau pour le modèle*).

Le principal **inconvénient** réside lui dans son **coût computationnel élevé** (*nécessitant d'entraîner et tester le modèle 99 fois*). Cependant, pour k NN qui ne nécessite aucun calcul lors de l'entraînement, et avec un faible nombre d'échantillons, cette limitation est largement atténuée.

B. Implémentation

Notre implémentation s'articule autour de deux modules Python.

Le module principal, *knn_classif.py*, encapsule l'intégralité de l'algorithme k NN dans une classe **ClassifKNN** qui expose une interface inspirée de la bibliothèque scikit-learn.

- La méthode **entraîner()** constitue la phase d'entraînement, bien qu'elle se limite à stocker les données. Concrètement, elle copie les matrices *X_train* (des features d'entraînement) et *y_train* (des étiquettes d'entraînement) dans les attributs de l'objet, ce qui les rend disponibles pour les prédictions ultérieures.
- Le cœur de l'algorithme réside dans la méthode **predire_un()** qui classe un unique échantillon test. Cette méthode commence par normaliser l'échantillon avec les mêmes paramètres que l'ensemble d'entraînement si nécessaire. Elle appelle ensuite la fonction **calculer_distance()** pour chaque échantillon d'entraînement, qui implémente la métrique de Minkowski. Les résultats sont stockés dans une liste de tuples contenant à la fois la distance et la classe correspondante. Un tri de cette liste par distance croissante permet d'identifier les *k* échantillons les plus proches. La décision finale s'effectue grâce à un objet **Counter** de la bibliothèque *collections* qui compte les occurrences de chaque classe parmi les *k* voisins, et retourne la classe la plus fréquente via la méthode **most_common()**. Cette méthode retourne également les distances et classes des *k* voisins, informations utiles pour des analyses ultérieures ou pour le calcul de probabilités d'appartenance.
- La méthode **predire()** généralise **predire_un()** à un ensemble d'échantillons en itérant sur chaque ligne de la matrice d'entrée.
- Enfin, **predire_proba()**, calcule des probabilités d'appartenance aux classes en transformant les votes des *k* voisins en proportions. Pour chaque échantillon test, elle compte combien de voisins appartiennent à chaque classe et divise par *k* pour obtenir des valeurs entre 0 et 1. Ces probabilités (approximatives) s'avèrent utiles pour construire les courbes précision-rappel (dans le fichier **courbe_pr.py** du dossier **utilitaires**!).

Le second module, **knn_test.py**, orchestre le processus complet d'évaluation en exploitant la structure *Leave-One-Out*. Il charge d'abord les données prétraitées sauvegardées lors de la partie de lecture des données sous format NumPy. Pour chaque méthode de description, il instancie un objet **LeaveOneOut** de *scikit-learn* qui génère les 99 paires d'indices train/test correspondant à chaque itération. À chaque itération, un nouveau classificateur **ClassifKNN** est créé, entraîné sur 98 échantillons, puis utilisé pour prédire la classe du 99e échantillon exclu. Les prédictions accumulées sont ensuite comparées aux vraies classes pour calculer la matrice de confusion via la fonction **confusion_matrix()** de *scikit-learn*, le taux de reconnaissance global, et un rapport détaillé par classe via **classification_report()** qui fournit précision, rappel et F-mesure pour chacune des neuf classes.

C. Résultats

L'évaluation des cinq méthodes de description avec **kNN** (*k=5*, distance euclidienne, normalisation activée) révèle une hiérarchie claire des performances :

Le descripteur **GFD** domine nettement avec un taux de reconnaissance de **85,86%** et un F1-score de 0,860, confirmant la pertinence de l'approche fréquentielle pour ce type de problème. Cette supériorité s'explique par sa capacité à encoder les contours dans le domaine fréquentiel en combinant informations radiales et angulaires, ce qui le rend particulièrement robuste aux distorsions et variations géométriques présentes dans le dataset. Les principales erreurs de GFD concernent la confusion entre classes 3 et 8, probablement due à des similarités de contour entre silhouettes humaines et certains animaux. (*Notons qu'avec la distance de Manhattan, GFD améliore légèrement son taux de reconnaissance.*)

Le descripteur **F0** se positionne en seconde place avec **74,75%** de reconnaissance, suggérant que ce dernier capture des propriétés géométriques complémentaires à GFD, notamment liées à la déformabilité des formes. Cependant, la classe 1 n'est correctement identifiée que dans 36,4% des cas, avec 5 échantillons confondus avec la classe 8, révélant des difficultés sur les formes occultées.

Les descripteurs **E34**, **SA** et **F2** affichent des performances médiocres comprises entre 55,6% et 62,6%. E34 montre un profil contrasté avec une reconnaissance parfaite de la classe 3 mais seulement 27,3% pour les classes 1 et 9, confirmant les limites des descripteurs basés uniquement sur les moments d'ordre 2. SA s'effondre sur certaines classes : la classe 2 n'obtient que 9,09% de rappel avec 7 échantillons sur 11 mal classés. F2 présente les résultats les plus faibles avec des échecs sur les classes 5 et 7 (9,09% de rappel chacune) et une forte dispersion des erreurs sur l'ensemble des classes.

L'**analyse comparative** révèle plusieurs tendances importantes. Aucun descripteur n'atteint la perfection, confirmant la complexité du dataset BDshape avec ses formes occultées, partiellement représentées et déformées. Certaines classes comme la 2 et la 9 sont systématiquement bien reconnues, tandis que les classes 1, 5 et 7 posent des difficultés récurrentes quelle que soit la méthode employée. Des patterns de **confusion** récurrents émergent

entre certaines paires de classes (1-8, 3-5), indépendamment du descripteur utilisé, suggérant des ambiguïtés géométriques intrinsèques au problème. Ces observations motivent l'exploration ultérieure d'une approche de fusion combinant plusieurs descripteurs pour exploiter leurs complémentarités respectives.

L'évaluation restreinte aux **cinq premières classes** confirme la supériorité de GFD avec **85,45%** de reconnaissance moyenne, suivi par F0 avec 72,73% et E34 avec 63,64%.

– Résultats pour GFD :

Taux de reconnaissance : 85.86%

Matrice de confusion :

```
[[10 0 0 0 0 0 0 1 0]
 [ 0 11 0 0 0 0 0 0 0]
 [ 0 0 7 0 0 0 0 4 0]
 [ 0 1 0 10 0 0 0 0 0]
 [ 0 0 0 0 9 0 0 2 0]
 [ 0 0 1 0 0 10 0 0 0]
 [ 2 0 0 0 0 1 0 8 0]
 [ 0 0 0 0 0 0 0 11 0]
 [ 0 0 0 2 0 0 0 0 9]]
```

Rapport de classification :

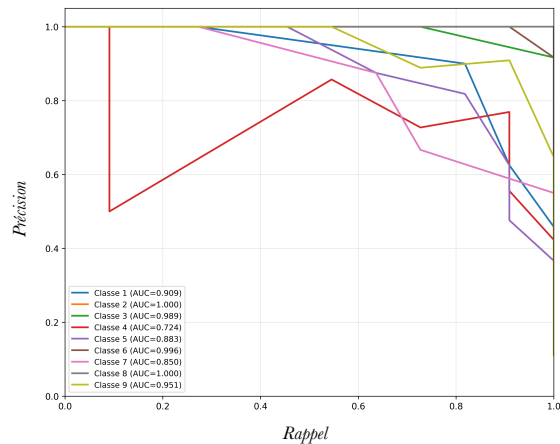
	precision	recall	f1-score	support
Classe 1	0.833333	0.909091	0.869565	11.000000
Classe 2	0.916667	1.000000	0.956522	11.000000
Classe 3	0.875000	0.636364	0.736842	11.000000
Classe 4	0.833333	0.909091	0.869565	11.000000
Classe 5	0.900000	0.818182	0.857143	11.000000
Classe 6	1.000000	0.909091	0.952381	11.000000
Classe 7	1.000000	0.727273	0.842105	11.000000
Classe 8	0.611111	1.000000	0.758621	11.000000
Classe 9	1.000000	0.818182	0.900000	11.000000
accuracy	0.858586	0.858586	0.858586	0.858586
macro avg	0.885494	0.858586	0.860305	99.000000
weighted avg	0.885494	0.858586	0.860305	99.000000

Tableau récapitulatif :

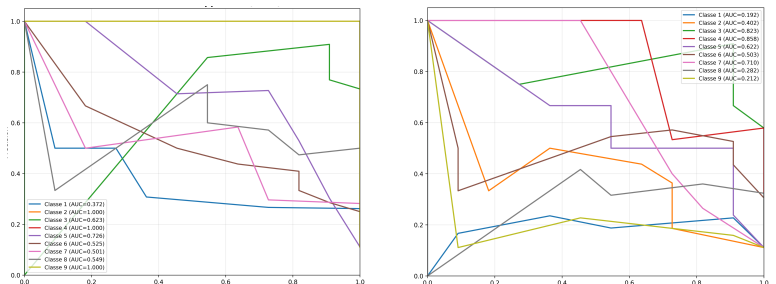
Méthode	Taux de reconnaissance (%)	F1-score macro
E34	62.626263	0.607294
GFD	85.858586	0.860305
SA	59.595960	0.558221
F0	74.747475	0.736615
F2	55.555556	0.536197

Résultats sauvegardés dans 'resultats_knn.npy'

Extraits de résultats : GFD et Tableau récapitulatif



Courbe Précision-Rappel pour GFD, avec k=10



Courbe Précision-Rappel pour F0 et E34, avec k=10 et respectivement un AUC moyen de 0,70 et 0,52

Enfin, l'analyse de la **courbe précision-rappel** pour le descripteur **GFD** permet d'évaluer la capacité du classificateur à distinguer chaque classe de toutes les autres selon différents seuils de décision. La **précision** mesure la proportion d'exemples correctement classés parmi ceux attribués à une classe donnée, tandis que le **rappel** quantifie la proportion d'exemples de cette classe effectivement retrouvés. L'aire sous la courbe **AUC** synthétise ces performances en une métrique unique comprise entre 0 et 1, où des valeurs supérieures à 0,9 indiquent une excellente capacité de discrimination.

Les résultats obtenus via le module **courbe_pr.py** (avec k=10, comme demandé dans le sujet) confirment l'efficacité de GFD avec un **AUC moyen** de **0,922**. Deux classes atteignent un score parfait de 1,0 : la classe 2 (*lapins*) et la classe 8 (*animaux hétérogènes*), démontrant une séparation parfaite avec les autres catégories. Les classes 3 (*silhouettes*) et 6 (*outils*) affichent également d'excellents scores respectifs de 0,989 et 0,996, confirmant la robustesse du descripteur face aux variations géométriques. La classe 4 (*avions*) présente le score le plus bas à 0,724, ce qui reste dans la catégorie "bon" et s'explique par les occlusions mentionnées dans le dataset.

Ces résultats valident quantitativement la supériorité de l'approche fréquentielle pour ce problème de reconnaissance de formes.

IV. Classification par k -means

A. Méthodologie

L'algorithme des k -means constitue notre seconde approche de classification, se distinguant de k NN par sa nature **non supervisée**. Contrairement aux méthodes supervisées qui disposent des étiquettes de classes pendant l'apprentissage, k -means cherche à découvrir une **structure naturelle** dans les données sans aucune connaissance préalable des catégories. L'algorithme organise les échantillons en k groupes homogènes appelés clusters, en supposant que les données présentent une organisation en nuages de points distincts dans l'espace des caractéristiques.

Le principe fondamental de k -means repose sur la **minimisation** d'un **critère d'erreur quadratique** (également appelé **inertie**). Pour un ensemble de données réparties en k clusters, chaque cluster est caractérisé par son centre (ou *centroïde*). Le critère à minimiser s'exprime comme la somme des distances euclidiennes au carré entre chaque point et le centre de son cluster. Formellement, si l'on note les k centres comme un ensemble de vecteurs dans l'espace des caractéristiques, et si chaque point est assigné à l'un de ces centres, l'objectif consiste à trouver la configuration des centres et des assignations qui minimise cette erreur quadratique totale.

L'**algorithme itératif** de k -means (*spécifié dans l'annexe du sujet*) se décompose en quatre étapes principales.

- La première étape consiste à **initialiser** les **k centres** de manière aléatoire, généralement en sélectionnant k échantillons parmi les données disponibles. Cette initialisation aléatoire introduit une variabilité dans les résultats, ce qui justifie la stratégie de multi-initialisation adoptée.
- La deuxième étape **affecte chaque point** au **cluster** dont le centre est le plus proche selon la distance euclidienne, créant ainsi une partition de l'ensemble des données.
- La troisième étape **recalcule** les **centres de chaque cluster** comme la moyenne arithmétique de tous les points qui lui ont été assignés.
- Ces **étapes deux et trois** sont **répétées** alternativement jusqu'à satisfaction d'un critère de convergence, typiquement la stabilisation des centres entre deux itérations successives.

L'algorithme **converge** alors vers un minimum local plutôt que global, ce qui signifie que la solution finale dépend fortement de l'initialisation. Pour contourner cette limitation, la pratique standard consiste à exécuter l'algorithme avec plusieurs initialisations aléatoires différentes et à conserver la solution présentant l'inertie la plus faible, stratégie que nous avons implémentée avec **dix initialisations** par défaut.

Un aspect crucial de k -means dans notre contexte réside dans la différence entre **clustering** et **classification**. Le clustering produit des groupes sans sémantique préétablie : les k clusters obtenus ne correspondent pas nécessairement aux classes réelles du problème. Pour évaluer les performances de k -means sur notre dataset où nous connaissons les vraies classes, nous devons résoudre un problème d'**appariement** entre les clusters découverts et les classes réelles. Cette problématique s'apparente au problème d'affectation optimale, résolu par l'**Algorithme Hongrois**. Concrètement, nous construisons une matrice de coût où chaque entrée représente le nombre de points d'un cluster donné appartenant à une classe donnée, puis nous recherchons l'appariement qui maximise le nombre total de points correctement associés.

Le choix du **paramètre k** , fixé à 9 dans notre cas (*pour correspondre aux neuf classes*), est lié à notre objectif de comparer k -means à l'approche supervisée k NN. Cette contrainte transforme partiellement l'exercice en une évaluation de la capacité de k -means à redécouvrir spontanément la structure de classes présente dans les données, sans en avoir été informé préalablement.

Comme pour k NN, nous appliquons une **normalisation z-score** aux données. Cette étape est nécessaire car k -means utilise la distance euclidienne pour assigner les points aux clusters, et sans normalisation, les dimensions avec de grandes valeurs numériques (*comme $F0$ et $F2$*) domineraient complètement le calcul de distance.

B. Implémentation

L'implémentation de k -means s'articule autour du module ***kmeans_classif.py*** qui encapsule l'algorithme dans une classe ***ClassifKMeans***. Le constructeur accepte quatre paramètres essentiels : le nombre de clusters k (*fixé à 9 pour correspondre aux neuf classes*), le nombre maximum d'itérations pour éviter les boucles infinies (*par défaut 100*), le

nombre d'initialisations aléatoires différentes à tester (*par défaut 10*), et une graine aléatoire (*42 dans nos essais*) pour assurer la reproductibilité des expériences.

- La méthode principale ***entraîner()*** implémente fidèlement l'algorithme itératif spécifié dans l'annexe du sujet. Pour gérer le problème des minima locaux (*expliqué dans la sous-partie précédente*), elle exécute l'algorithme complet avec dix initialisations aléatoires différentes et conserve la solution présentant l'inertie la plus faible.
- À chaque initialisation, la méthode ***initialiser_centres()*** sélectionne aléatoirement k échantillons parmi les données pour servir de centres initiaux. Cette approche garantit que les centres de départ appartiennent à la région de l'espace occupée par les données réelles.
- La méthode ***assigner_clusters()*** implémente l'étape d'assignation en calculant la distance euclidienne entre chaque point et chacun des k centres, puis en attribuant chaque point au centre le plus proche. Cette opération génère une matrice de distances (*de dimension $n \times k$ où n représente le nombre d'échantillons*), et utilise la fonction ***argmin()*** de NumPy pour identifier efficacement le cluster optimal pour chaque point.
- La méthode ***mettre_a_jour_centres()*** réalise ensuite le re-calcul des centres en calculant le barycentre de tous les points appartenant à chaque cluster via la fonction ***mean()*** appliquée sur l'axe approprié. Une précaution importante consiste à gérer les clusters vides qui peuvent apparaître lors de certaines initialisations : dans ce cas, le centre est réinitialisé aléatoirement pour éviter une division par zéro.
- La convergence de l'algorithme est détectée par la méthode ***allclose()*** de NumPy qui compare les centres actuels avec les centres de l'itération précédente selon une tolérance numérique fixée. Cette approche évite les problèmes d'égalité stricte entre nombres flottants tout en garantissant une convergence effective.
- La méthode ***calculer_inertie()*** calcule le critère d'erreur quadratique en sommant les distances au carré entre chaque point et le centre de son cluster, fournissant ainsi une mesure quantitative de la qualité du partitionnement obtenu.
- Le module inclut également la fonction ***mapper_clusters_classes()*** implémentée en dehors de la classe ***ClassifKMeans***, qui résout le problème d'appariement entre clusters et classes réelles. Cette fonction construit une matrice de coût où chaque entrée représente le nombre négatif de points d'un cluster donné appartenant à une classe donnée. L'algorithme hongrois, implémenté par la fonction ***linear_sum_assignment()*** de SciPy, trouve ensuite l'appariement optimal qui maximise le nombre total de correspondances correctes. Cette étape de post-traitement est indispensable pour évaluer les performances de k -means dans notre contexte de classification supervisée, bien que l'algorithme lui-même n'utilise jamais cette information pendant son exécution.

Le module d'évaluation ***kmeans_test.py*** orchestre le processus complet en chargeant les données, en appliquant une normalisation z-score pour égaliser l'influence des différentes dimensions, puis en exécutant k -means sur chacune des cinq méthodes de description. Contrairement à k NN qui nécessitait une validation croisée Leave-One-Out, k -means est évalué directement sur l'ensemble complet des 99 échantillons puisque sa nature non supervisée rend inappropriée la notion de données de test séparées.

C. Résultats

Les résultats de ***k-means*** révèlent des performances globalement inférieures à celles de ***kNN***, avec des **taux de reconnaissance** s'échelonnant de 46,46% pour **E34** à 59,60% pour **F0**. Cette hiérarchie diffère significativement de celle observée avec k NN où **GFD** dominait à 85,86% : l'écart de performance entre les deux approches atteint jusqu'à 31 points pour cette méthode, illustrant la difficulté fondamentale de la classification **non supervisée** lorsque la structure géométrique naturelle des données ne correspond pas à la structure sémantique des classes définies.

L'analyse détaillée des **matrices de confusion** révèle plusieurs phénomènes. Pour GFD, trois classes obtiennent un **rappel** strictement nul (*classes 1, 3 et 8*), signifiant qu'aucun de leurs échantillons n'a été correctement identifié après le mapping optimal. Cette situation s'explique par la nature de **l'algorithme hongrois** qui recherche l'appariement global maximisant le nombre total de correspondances correctes. Concrètement, k -means a effectivement créé neuf clusters, mais leur contenu ne correspond pas aux classes réelles. Le cas de la classe 5 illustre ce compromis : elle obtient un rappel de 100%, signifiant que tous ses onze échantillons se trouvent dans le même cluster. Cependant, sa précision s'effondre à 32,4%, indiquant que ce cluster contient également la totalité des échantillons des classes 3 et 8. L'algorithme hongrois, contraint d'assigner ce cluster volumineux à une seule classe, l'attribue à la classe 5 car elle y est majoritaire, sacrifiant ainsi les classes 3 et 8 qui obtiennent donc un rappel nul. Ce phénomène démontre que k -means a regroupé trois classes **sémantiquement distinctes** (*silhouettes, mains*

et animaux) dans un même cluster géométrique, suggérant qu'elles partagent des **caractéristiques similaires** dans l'espace GFD malgré leurs **différences sémantiques**.

Le descripteur **F0** présente un profil légèrement plus équilibré avec 59,60% de reconnaissance, positionnant quatre classes au-dessus de 80% de rappel (*classes 2, 3, 4 et 9*). Cette performance supérieure à GFD en contexte non supervisé suggère que F0 génère un **espace de caractéristiques** où les classes forment des groupes plus naturellement séparés. Cependant, la classe 7 obtient à nouveau un rappel nul, ses échantillons étant dispersés principalement entre les clusters 5 et 6 selon la matrice de confusion. Cette difficulté récurrente sur la classe 7, observée également avec *k*NN dans une moindre mesure, révèle une **faiblesse du descripteur F0** pour cette catégorie plutôt qu'une limitation spécifique de *k*-means.

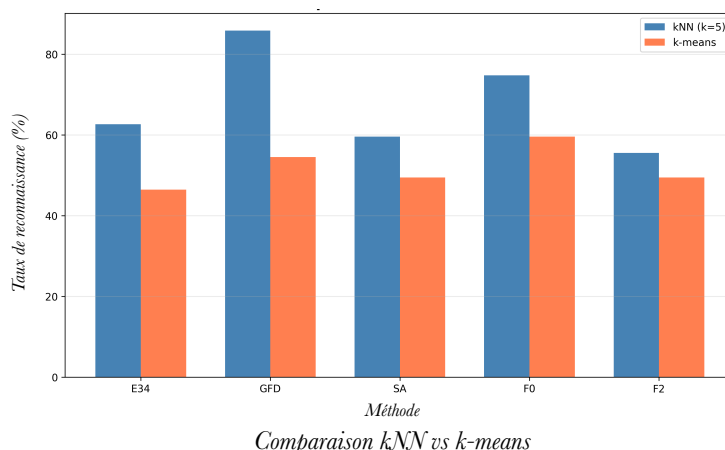
Les résultats de **E34** confirment l'insuffisance des descripteurs basés uniquement sur les moments d'ordre 2 pour capturer la complexité discriminante nécessaire. Avec seulement 46,46% de reconnaissance, E34 présente la matrice de confusion la plus dispersée, sans pattern de confusion dominant. Cette dispersion indique que les classes partagent des propriétés d'ellipticité tellement similaires qu'elles forment des nuages de points fortement chevauchants dans l'espace à seize dimensions défini par E34.

Enfin, au-delà de la distinction évidente entre **apprentissage supervisé** et **non supervisé**, plusieurs facteurs expliquent la moindre performance de **k-means** par rapport à *k*NN :

- Premièrement, *k*-means repose sur une **hypothèse géométrique** forte : les clusters doivent être approximativement sphériques avec des variances comparables. Or, les classes réelles de notre dataset représentent des formes arbitraires, des distributions allongées ou des densités variables que *k*-means ne peut pas capturer.
- Deuxièmement, *k*-means minimise l'**inertie intra-cluster**, c'est-à-dire la compacité des groupes, sans considération pour la séparabilité inter-classes. Un **partitionnement** peut être optimal du point de vue de l'inertie tout en mélangeant plusieurs classes sémantiques dans un même cluster si celles-ci sont géométriquement proches.
- Troisièmement, les classes de notre dataset sont définies par des **critères sémantiques externes** (*lapins, avions, silhouettes*) qui ne correspondent pas nécessairement à des **groupements naturels** dans l'espace des caractéristiques : *k*NN bénéficie des étiquettes pour apprendre directement quelles différences géométriques sont pertinentes pour la classification, tandis que *k*-means doit découvrir une structure. Si deux classes sont sémantiquement distinctes mais géométriquement proches, *k*-means les fusionnera inévitablement.

Résultats pour F0 :				
Taux de reconnaissance : 59.60%				
Inertie: 1729.23				
Matrice de confusion :				
[[4 0 0 0 1 1 0 5 0]				
[0 11 0 0 0 0 0 0 0]				
[0 0 10 0 1 0 0 0 0]				
[2 0 0 9 0 0 0 0 0]				
[0 0 7 0 4 0 0 0 0]				
[0 0 0 0 2 9 0 0 0]				
[0 0 1 0 1 7 0 2 0]				
[0 0 0 0 5 1 0 5 0]				
[0 0 0 0 0 0 4 0 7]]				
Rapport de classification :				
	precision	recall	f1-score	support
Classe 1	0.666667	0.363636	0.476588	11.00000
Classe 2	1.000000	1.000000	1.000000	11.00000
Classe 3	0.555556	0.909091	0.689655	11.00000
Classe 4	1.000000	0.818182	0.900000	11.00000
Classe 5	0.285714	0.363636	0.320000	11.00000
Classe 6	0.500000	0.818182	0.620690	11.00000
Classe 7	0.000000	0.000000	0.000000	11.00000
Classe 8	0.416667	0.454545	0.434783	11.00000
Classe 9	1.000000	0.636364	0.777778	11.00000
accuracy	0.595960	0.595960	0.595960	0.59596
macro avg	0.602734	0.595960	0.579277	99.00000
weighted avg	0.602734	0.595960	0.579277	99.00000
Tableau récapitulatif :				
Méthode	Taux de reconnaissance (%)	Inertie	F1-score macro	
E34	46.464646	64.108118	0.438182	
GFD	54.545455	5249.081768	0.489593	
SA	49.494949	3573.625402	0.429731	
F0	59.595960	1729.228577	0.579277	
F2	49.494949	326.479034	0.480889	

Extraits de résultats : F0 et Tableau récapitulatif



Résultats pour GFD :	
Taux de reconnaissance : 54.55%	
Inertie: 5249.08	
Matrice de confusion :	
[[0 0 0 0 1 0 10 0 0]	
[0 11 0 0 0 0 0 0 0]	
[0 0 0 0 11 0 0 0 0]	
[0 0 0 9 0 0 0 2 0]	
[0 0 0 0 11 0 0 0 0]	
[4 0 0 1 0 6 0 0 0]	
[0 0 0 0 0 0 11 0 0]	
[0 0 0 0 11 0 0 0 0]	
[0 0 4 1 0 0 0 0 6]]	

Résultats pour E34 :	
Taux de reconnaissance : 46.46%	
Inertie: 64.11	
Matrice de confusion :	
[[4 1 0 0 4 0 0 2 0]	
[3 5 0 0 1 1 1 0 0]	
[0 0 10 0 0 0 1 0 0]	
[0 0 0 6 0 4 0 0 1]	
[3 0 0 0 7 0 0 1 0]	
[0 4 0 0 0 7 0 0 0]	
[1 1 6 0 0 0 3 0 0]	
[0 1 0 0 6 0 0 4 0]	
[0 1 1 0 3 0 2 4 0]]	

Extraits de résultats : GFD et E34

V. Classification par vote majoritaire

A. Méthodologie

L'approche par vote majoritaire constitue notre troisième et dernière méthode de classification, fondamentalement différente des deux précédentes par sa **nature composite**. Plutôt que d'utiliser un seul espace de caractéristiques pour prendre une décision, cette approche combine les prédictions de plusieurs classificateurs indépendants opérant chacun sur une représentation différente des données.

Le principe du vote majoritaire est le suivant : pour **classifier** un échantillon donné, on sollicite plusieurs classificateurs de base qui émettent chacun une prédiction, puis la classe finale est déterminée par la classe ayant obtenu **le plus de votes**. La force de cette approche réside dans la diversité des descripteurs : lorsqu'un descripteur échoue à cause de similarités géométriques dans son espace spécifique, un autre basé sur des propriétés distinctes peut compenser cette faiblesse.

L'implémentation que nous avons retenue respecte le protocole de validation **Leave-One-Out**. Une approche alternative aurait consisté à combiner les prédictions individuelles calculées lors des étapes précédentes. Nous avons écarté cette stratégie, car manquait de rigueur en mélangeant les phases d'évaluation. Notre implémentation garantit alors que pour chaque itération, cinq classificateurs sont entraînés indépendamment avant que leurs votes ne soient agrégés, assurant une stricte séparation entre apprentissage et test.

Nous avons évalué deux configurations :

- Le vote intégral avec les **5 méthodes** : une approche où **chaque descripteur a un poids égal**.
- Le vote sélectif avec le **top 3** : une combinaison restreinte aux **trois meilleures méthodes** (*GFD*, *F0*, *E34*).

B. Implémentation

Le module principal, *vote_classif.py*, définit la classe *ClassifVoteMajoritaire* qui encapsule la logique de classification, tandis que le module d'évaluation *vote_test.py* orchestre le processus complet de validation croisée et de comparaison des configurations.

La classe *ClassifVoteMajoritaire* implémente un pattern de conception de type compositeur qui agrège plusieurs classificateurs de base sans connaître leurs détails d'implémentation. Le constructeur accepte trois paramètres essentiels : une classe de classificateur de base (*dans notre cas* *ClassifKNN*), la liste des méthodes de description à combiner, et un indicateur de normalisation. Cette conception générique permet en théorie d'utiliser n'importe quel type de classificateur de base.

- La méthode *entraîner()* constitue le cœur de la phase d'entraînement du classificateur. Elle reçoit un dictionnaire structuré associant chaque méthode de description à ses données d'entraînement (*matrice de caractéristiques et vecteur de classes*). Pour chaque méthode, elle instancie un nouveau classificateur *kNN*, l'entraîne sur les données correspondantes, puis stocke à la fois le classificateur entraîné et les données d'entraînement dans un dictionnaire interne. Cette conservation des données d'entraînement facilite l'accès ultérieur lors des prédictions dans le contexte de la validation Leave-One-Out où les échantillons tests proviennent directement de ces ensembles. La structure de données résultante forme un registre de classificateurs spécialisés, chacun expert dans une représentation particulière des formes.
- La méthode *predire()* implémente le processus de vote majoritaire proprement dit. Elle reçoit un dictionnaire associant chaque méthode aux échantillons à classifier, ce qui permet de traiter simultanément plusieurs échantillons tout en maintenant la correspondance entre les différentes représentations d'un même objet. Pour chaque échantillon, la méthode itère sur tous les classificateurs de base, sollicite leur prédiction via leur méthode *predire()* (*dans notre cas, la méthode *predire()* de *kNN**), et collecte les votes dans une liste. L'agrégation finale s'effectue via l'objet Counter de la bibliothèque collections qui compte efficacement les occurrences de chaque classe et identifie la classe majoritaire grâce à la méthode *most_common()*. En cas d'égalité parfaite entre deux classes, la première dans l'ordre lexicographique est sélectionnée, bien que cette situation soit rare avec cinq votants.

Le module d'évaluation *vote_test.py* implémente Leave-One-Out en orchestrant l'interaction entre le classificateur et les données. Pour chaque des 99 itérations, le script construit dynamiquement les ensembles d'entraînement pour chacune des méthodes en excluant l'échantillon test courant, instancie un nouveau *ClassifVoteMajoritaire*, l'entraîne sur ces ensembles, puis sollicite sa prédiction pour l'échantillon test.

C. Résultats

Les résultats du **vote majoritaire** confirment l'hypothèse de **complémentarité** des descripteurs tout en révélant l'importance de la sélection judicieuse des contributeurs au vote. La configuration combinant les trois meilleures méthodes **GFD**, **F0**, et **E34** atteint un taux de reconnaissance de **88,89%**, établissant ainsi la meilleure performance observée dans l'ensemble de nos expérimentations et surpassant de trois points la meilleure méthode individuelle (*GFD* à 85,86%). Ce gain significatif valide l'approche de fusion de descripteurs et démontre que les différentes représentations géométriques capturent effectivement des informations discriminantes complémentaires.

La configuration utilisant les **cinq méthodes** sans distinction obtient un taux de **82,83%**, soit près de trois points de moins que GFD utilisé seul. Ce résultat s'explique par un effet de **pollution du vote** : les classificateurs basés sur **SA** (59,60% en individuel) et **F2** (55,56% en individuel) émettent des prédictions erronées avec une fréquence suffisamment élevée pour dégrader le consensus.

Considérons par exemple le cas d'un échantillon de la classe 1 : si GFD, F0 et E34 prédisent correctement la classe 1 tandis que SA et F2 se trompent en prédisant respectivement les classes 5 et 8, le vote donne trois voix pour la classe 1 contre deux voix dispersées, assurant la victoire de la classe correcte.

Cependant, dans les situations où GFD **hésite**, le bruit introduit par SA et F2 peut faire basculer le vote vers une classe erronée. Ainsi, la diversité des classificateurs n'est bénéfique que si leur qualité individuelle dépasse un certain seuil.

- Méthodes combinées : GFD, E34, F0				
Taux de reconnaissance : 88.89%				
Matrice de confusion :				
[[8 0 0 0 0 0 0 3 0]				
[0 11 0 0 0 0 0 0 0]				
[0 0 11 0 0 0 0 0 0]				
[0 0 0 11 0 0 0 0 0]				
[0 0 0 0 9 0 0 2 0]				
[0 0 1 0 0 10 0 0 0]				
[2 0 0 0 1 0 8 0 0]				
[0 0 0 0 0 0 0 11 0]				
[0 0 0 2 0 0 0 0 9]]				
Rapport de classification :				
	precision	recall	f1-score	support
Classe 1	0.800000	0.727273	0.761905	11.000000
Classe 2	1.000000	1.000000	1.000000	11.000000
Classe 3	0.916667	1.000000	0.956522	11.000000
Classe 4	0.846154	1.000000	0.916667	11.000000
Classe 5	0.900000	0.818182	0.857143	11.000000
Classe 6	1.000000	0.909091	0.952381	11.000000
Classe 7	1.000000	0.727273	0.842105	11.000000
Classe 8	0.687500	1.000000	0.814815	11.000000
Classe 9	1.000000	0.818182	0.900000	11.000000
accuracy	0.888889	0.888889	0.888889	0.888889
macro avg	0.905591	0.888889	0.889060	99.000000
weighted avg	0.905591	0.888889	0.889060	99.000000

Extraits de résultats : Top 3

L'analyse de la **matrice de confusion** pour la configuration **Top 3** révèle de très bonnes performances sur la majorité des classes, et en particulier pour quatre classes atteignant une reconnaissance parfaite avec un **rappel de 100%** : les classes 2 (*lapins*), 3 (*silhouettes*), 4 (*avions*) et 8 (*animaux*). Ce résultat contraste fortement avec les performances de GFD seul qui n'atteignait la perfection que sur deux classes. La complémentarité des descripteurs se manifeste particulièrement sur la classe 4 où GFD seul obtenait 90,9% de rappel avec une confusion occasionnelle avec la classe 8. L'ajout de F0 et E34 au vote permet de corriger ces erreurs marginales : dans les cas ambigus pour GFD, au moins un des deux autres descripteurs fournit l'information discriminante nécessaire. Cette synergie démontre que les trois méthodes capturent des **aspects géométriques** suffisamment différents pour que leurs **zones d'incertitude** ne se chevauchent pas systématiquement.

La **comparaison globale** avec les autres approches testées positionne le vote majoritaire Top 3 comme la méthode **optimale** pour ce problème. Avec **88,89%** de taux de reconnaissance, elle surpasse **kNN** (55,56% à 85,86%), et devance **k-means** de manière significative (46,46% à 59,60%). Le gain de trois points par rapport à **GFD** seul peut paraître modeste, mais représente une **réduction de 21%** du taux d'erreur (de 14,14% à 11,11%). L'analyse révèle également que la **combinaison aveugle** de tous les descripteurs disponibles n'est pas une stratégie optimale. La sélection des contributeurs au vote, basée sur leurs performances individuelles, s'avère bien meilleure pour maximiser le bénéfice de l'approche ensembliste. Ce résultat suggère qu'une approche de pondération des votes proportionnelle aux performances individuelles pourrait potentiellement améliorer encore les résultats, bien que nous n'ayons pas exploré cette piste dans le cadre de ce projet.

VI. Conclusions

Les résultats révèlent une hiérarchie claire tant au niveau des méthodes de description qu'au niveau des approches de classification. Parmi les descripteurs, **GFD** s'impose comme le plus performant avec **85,86%** de reconnaissance en classification supervisée **kNN**, confirmant la pertinence de l'approche fréquentielle pour capturer les propriétés discriminantes des contours. Le descripteur **F0**, basé sur les forces constantes, atteint une performance de **74,75%**, démontrant que les propriétés de déformation constituent également une source d'information pertinente. Le descripteur **E34**, fondé sur les moments d'ordre 2, obtient **62,63%**, révélant que l'ellipticité seule ne suffit pas à discriminer efficacement les neuf classes. Les descripteurs **SA** et **F2** affichent les performances les plus modestes avec respectivement **59,60%** et **55,56%**, suggérant des limitations intrinsèques de ces représentations pour ce problème spécifique.

L'approche supervisée **kNN** surpasse systématiquement **k-means** sur toutes les méthodes de description, avec des écarts allant de six à trente-et-un points de pourcentage. Cette différence illustre l'avantage de l'information d'étiquetage : **kNN** apprend directement des **exemples annotés** pour construire ses frontières de décision, tandis que **k-means** doit découvrir une **structure géométrique naturelle** qui ne correspond pas nécessairement aux catégories sémantiques prédéfinies. Les résultats de **k-means** bien que significativement inférieurs avec un maximum de **59,60%** pour **F0**, restent néanmoins informatifs en révélant que certaines méthodes de description créent des espaces où les classes forment des groupements naturels plus cohérents que d'autres.

Le **vote majoritaire** représente l'aboutissement de notre démarche en exploitant la **complémentarité** des différentes représentations. La configuration combinant les trois meilleures méthodes (**GFD**, **F0**, **E34**) atteint un taux de reconnaissance de **88,89%**, surpassant de trois points la meilleure méthode individuelle et établissant ainsi la performance optimale de l'ensemble de nos expérimentations. Ce résultat valide l'hypothèse selon laquelle différents descripteurs capturent des **aspects géométriques complémentaires** et que leur fusion permet d'exploiter plus complètement **l'information discriminante**. L'analyse comparative révèle également qu'une sélection judicieuse des contributeurs au vote s'avère essentielle : la configuration utilisant les cinq méthodes sans distinction obtient seulement **82,83%**, inférieure à **GFD** seul, démontrant qu'ajouter des classificateurs faibles peut **polluer** le consensus plutôt que l'améliorer.

L'analyse des **matrices de confusion** révèle des patterns récurrents indépendants de l'approche de classification. Certaines classes comme la 2 (*lapins*) et la 3 (*silhouettes*) sont systématiquement bien reconnues par tous les classificateurs, suggérant des caractéristiques géométriques fortement discriminantes. À l'inverse, les classes 1 et 7 posent des difficultés persistantes avec des confusions fréquentes, indiquant des ambiguïtés géométriques que nos descripteurs ne parviennent pas à résoudre complètement. Le phénomène de confusion systématique entre certaines paires de classes (*notamment 1-8 et 3-5*) transcende les différentes représentations et méthodes testées, suggérant des limitations fondamentales liées à la nature même du dataset ou aux critères de définition des classes.

Plusieurs **limites** de notre étude sont à mentionner. La **taille restreinte** du dataset limite la robustesse statistique de nos conclusions et pourrait expliquer certaines variations de performance. Aussi, le protocole **Leave-One-Out**, bien qu'adapté aux petits ensembles, présente une variance d'estimation plus élevée que des approches sur des datasets plus conséquents.

Plusieurs **perspectives d'amélioration** se dégagent de ce travail. L'exploration de méthodes de description plus récentes basées sur l'apprentissage profond pourrait potentiellement améliorer les performances. L'implémentation d'un **vote pondéré**, où chaque classificateur contribue proportionnellement à sa performance individuelle, pourrait améliorer notre approche de fusion. L'application de techniques de sélection de caractéristiques au sein de chaque descripteur pourrait permettre d'identifier les dimensions les plus discriminantes et d'améliorer la séparabilité des classes.

En définitive, ce projet démontre que la combinaison judicieuse de représentations complémentaires via des méthodes d'ensemble constitue une stratégie efficace pour la reconnaissance de formes, tout en soulignant l'importance fondamentale de l'information supervisée dans les tâches de classification.

VII. Sources

Les principales sources consultées et utilisées pour le développement et la compréhension du projet sont listées ci-dessous via des liens hyper-textes.

A. Références bibliographiques

Documents de Cours de Reconnaissance des Formes
Documents de Cours de Traitement Numériques des Données
Projet de CBIR de L3

B. Ressources en ligne

Classification **kNN** :

DataCamp : Classification par K-Nearest Neighbors

scikit-learn : KNeighborsClassifier

Medium : Implementing the k-Nearest Neighbors (KNN) Algorithm

Classification **k-means** :

DataCamp : k-Means Clustering

scikit-learn : KMeans

IBM Think : Qu'est-ce que l'apprentissage ensembliste

LeMagIT : Algorithme K-means expliqué

Classification **vote majoritaire** :

Inside Machine Learning : Méthodes d'Ensemble

DataCamp : Classification dans l'apprentissage automatique

Bibliothèques :

Documentation NumPy

Documentation scikit-learn

Documentation SciPy

Documentation Matplotlib

Documentation pandas

Algorithmes :

Wikipédia : Algorithme Hongrois

scikit-learn : Leave-One-Out